



MANUAL TÉCNICO

Simulador de CallCenter

Emily Maritza Tepeu Guacamaya

202402955

Práctica Única

LFP B-

TABLA DE CONTENIDO

INTRODUCCIÓN	4
REQUISITOS DEL SISTEMA	5
Requisitos Hardware	5
Procesador	5
Memoria RAM	5
Almacenamiento	5
Sistema Operativo	5
Requisitos Software	5
Node.js	5
NPM	5
IDE/Editor	5
Terminal/Consola	6
Dependencias	6
Conceptos de javascript	7
Estructuras de Control	7
Declaración de Variables	7
Estructura de Datos	7
Métodos	7
conceptos de html	8
Estructura Básica	8
Metadatos y Estilos	8
Elementos de Contenido	8
eSTRUCTURA DEL PROYECTO	9
CallCenter.js	9
Clase Principal	9
Método para cargar archivos CSV	9
Método para procesar los campos	9
Validación del ID	10
Contador de Estrellas y Clasificación	10
Guardado del Operador y cliente	10
Registro de la llamada	11
Clasificación de llamada según las estrellas	11
Contador de estrellas en la cadena	11

Funciones de Exportación	12
Exportar Historial	12
Exportar Listado de Operadores/Clientes	12
Exportar Rendimiento de Operadores	12
Porcentaje de Clasificación	13
Menú Principal	14
Ejecución del programa	15
Reportes.js	16
generarHistorial(llamadas)	16
generarOperadores(operadores)	16
generarClientes(clientes)	17
generarRendimientoOp()	17
guardarHTML(nombreArchivo, contenido)	18
Estructura HTML	19
Plantilla	19
Datos	19
Diagrama de flujo	¡Error! Marcador no definido.
Diagrama de clases	2
CONSIDERACIONES TÉCNICAS	1
¿Qué formato debe tener el archivo csv para ser procesado?	1
¿Qué pasa si el archivo csv no cumple con el formato requerido?	1
¿El sistema requiere encabezados en el archivo csv?	1
¿El sistema depende de paquetes externos?	1
¿Los reportes incluyen toda la información procesada?	1

INTRODUCCIÓN

Este sistema de gestión de CallCenter está desarrollado con Node.js y permite procesar, analizar y generar reportes de llamadas telefónicas a partir de archivos CSV. El sistema carga datos de operadores, clientes y calificaciones de llamadas, y genera reportes en formato HTML para visualización y análisis.

Alcance y Funcionalidades:

- Carga de registros de llamadas desde archivos CSV
- Exportación de reportes en formato HTML
- Análisis de desempeño por operador individual
- Estadísticas de clasificación de llamadas
- Gestión de datos de operadores y clientes
- Interfaz de consola interactiva

REQUISITOS DEL SISTEMA

Requisitos Hardware

Antes de ejecutar el programa, asegúrese de cumplir con los siguientes requisitos:

Procesador

- Mínimo: Dual-Core 1.5 GHz
- Recomendado: Quad-core 2.0 GHz+

Memoria RAM

- Mínimo: 4 GB
- Recomendado: 8 GB

Almacenamiento

- Mínimo: 500 MB libres
- Recomendado: 1 GB libre

Sistema Operativo

- Mínimo: Windows 10 / Ubuntu 18.04 / macOS 10.14
- Recomendado: Windows 11 / Ubuntu 20.04 / macOS 12+

Requisitos Software

Node.js

- Versión 16.0.0 o superior

NPM

- Incluido con Node.js

IDE/Editor

- Visual Studio Code
- WebStorm
- Sublime Text

Terminal/Consola

- PowerShell
- Command Prompt
- Terminal de Linux / Max

Dependencias

```
// Dependencias nativas de Node.js  
const fs = require("fs");           // Manejo de archivos del sistema  
const readline = require("readline"); // Entrada/salida por consola
```

CONCEPTOS DE JAVASCRIPT

Estructuras de Control

- **try-catch:** Manejo de errores, intenta ejecutar el código y captura errores
- **throw new Error()** : Lanza un error personalizado
- **for-of:** Bucle para iterar sobre elementos de arrays
- **switch-case:** Estructura para múltiples condiciones

Declaración de Variables

- **const:** Variable constante (no se puede reasignar)
- **let:** Variable que puede cambiar su valor
- **this:** Referencia al objeto actual de la clase

Estructura de Datos

- **Map():** Almacenamiento de clave-valor
- **Array:** Lista ordenada de elementos
- **Array.from():** Convierte Map/Set a Array

Métodos

- **split():** Divide string en array
- **filter():** Filtra elementos de array
- **sort():** Ordena un array
- **push():** Agrega elemento a un array
- **set():** Agrega elemento a Map
- **get():** Obtiene elemento de Map
- **has():** Verifica existencia en Map

CONCEPTOS DE HTML

Estructura Básica

- **<!DOCTYPE html>**: Declara tipo de documento
- **<html>**: Contenedor raíz
- **<head>**: Metadatos del documento
- **<body>**: Cuerpo del documento

Metadatos y Estilos

- **<meta charset="UTF-8">**: Codificación de caracteres
- **<title>**: Título del documento
- **<style>**: Estilos CSS embebidos

Elementos de Contenido

- **<h1>**: Encabezado principal
- **<table>**: Tabla de datos
- **<thead>**: Encabezado de la tabla
- **<tbody>**: Cuerpo de la tabla
- **<tr>**: Fila de la tabla
- **<th>**: Celda de encabezado
- **<td>**: Celda de datos
- **<p>**: Párrafos o textos informáticos adicionales

ESTRUCTURA DEL PROYECTO

CallCenter.js

Clase Principal

Este código define la estructura central del sistema, usando un array en "llamadas" para almacenar todas las llamadas procesadas. También dos mapas para "operadores" y "clientes", optimizando búsquedas por ID.

```
class CallCenter {  
  constructor() {  
    this.llamadas = [];  
    this.operadores = new Map();  
    this.clientes = new Map();  
  }  
}
```

Método para cargar archivos CSV

Su función es leer y procesar un archivo csv, leyendo el archivo de forma sincrónica y dividiendo el contenido por líneas filtrando las vacías.

```
cargarArchivo(ruta){  
  try{  
    const datos = fs.readFileSync(ruta, "utf-8");  
    const lineas = datos.split('\n').filter(linea => linea.trim() !== "");  
  }  
}
```

Método para procesar los campos

Divide una línea csv en campos individuales. Utiliza un parser "dividirLineaCSV" que maneja comas dentro de comillas.

```
try{  
  const campos = this.dividirLineaCSV(linea);  
  
  if(campos.length < 5){  
    console.log(`Línea ${i+1} ignorada: formato incorrecto`);  
    continue;  
  }  
}
```

Validación del ID

Valida que los IDs sean números, si algún ID no es un número se ignora la línea y se continua con la siguiente.

```
if (isNaN(idOperador) || isNaN(idCliente)) {
  console.log(`Línea ${i+1} ignorada: ID no valido`);
  continue;
}
```

Contador de Estrellas y Clasificación

Convierte la cadena de estrellas en un número y clasifica la llamada.

```
const calificacion = this.contarEstrellas(estrellasStr);
const clasificacion = this.clasificarLlamada(calificacion);
```

Guardado del Operador y cliente

Almacena operadores y clientes en sus respectivos mapas. Si el operador/cliente no existe, se crea un nuevo registro evitando duplicados por ID.

```
//GUARDAR OPERADOR
if(!this.operadores.has(idOperador)){
  this.operadores.set(idOperador, {
    id: idOperador,
    nombre: nombreOperador,
    llamadasAtendidas: 0
  });
}

//GUARDAR CLIENTE
if(!this.clientes.has(idCliente)){
  this.clientes.set(idCliente, {
    id: idCliente,
    nombre: nombreCliente
  });
}
```

Registro de la llamada

Agrega la llamada procesada al array "llamadas", almacenando todos los datos relevantes para luego actualizar el contador de llamadas atendidas según el operador.

```
//REGISTRAR LA LLAMADA
this.llamadas.push({
  idOperador,
  nombreOperador,
  calificacion,
  clasificacion,
  idCliente,
  nombreCliente
});

//INCREMENTAR EL CONTADOR DE LLAMADAS
this.operadores.get(idOperador).llamadasAtendidas++;
registrosCargados++;
```

Clasificación de llamada según las estrellas

Asigna una categoría basada en su calificación (Buena, Media, Mala).

```
clasificarLlamada(calificacion){
  if(calificacion >= 4) return "Buena";
  if(calificacion >= 2) return "Media";
  return "Mala";
}
```

Contador de estrellas en la cadena

Cuenta cuántas "X" hay en la cadena, estas están divididas por ";"

```
contarEstrellas(estrellasStr){
  const partes = estrellasStr.split(";");
  let count = 0;

  for(const parte of partes){
    if(parte.trim().toLowerCase() === "x"){
      count++;
    }
  }

  return count;
}
```

Funciones de Exportación

Exportar Historial

Genera un reporte HTML con todas las llamadas empleando el módulo "reportes.js" para crear una tabla con el historial.

```
exportarHistorialHTML(){
  if(this.llamadas.length === 0){
    console.log("\nNo hay llamadas registradas");
    return false;
  }
}
```

Exportar Listado de Operadores/Clientes

Genera reportes HTML de operadores/clientes, creando tablas con los datos almacenados en los mapas. En el caso de no haber ninguno, muestra un error.

```
exportarOperadoresHTML(){
  if(this.operadores.size === 0){
    console.log("\nNo hay operadores registrados");
    return false;
  }
}
```

```
exportarClientesHTML(){
  if(this.clientes.size === 0){
    console.log("\nNo hay clientes registrados");
    return false;
  }
}
```

Exportar Rendimiento de Operadores

Genera un reporte de rendimiento con porcentajes de atención. Calcula el porcentaje de llamadas atendidas por cada operador respecto al total.

```
exportarRendimientoHTML(){
  if(this.operadores.size === 0){
    console.log("\nNo hay operadores registrados");
    return false;
  }
}
```

Porcentaje de Clasificación

Muestra por consola el porcentaje de llamadas buenas, medias y malas para cada operador.

```
mostrarPorcentaje(){
  if(this.llamadas.length === 0){
    console.log("\nNo hay llamadas registradas");
    return false;
  }
}
```

Donde primeros e inicializan los contadores para luego clasificar según la clasificación y el operador.

```
//INICIALIZAR ESTADISTICAS PARA CADA OPERADOR
for (const [id, operador] of this.operadores) {
  estadisticasPorOperador[id] = {
    nombre: operador.nombre,
    buenas: 0,
    medias: 0,
    malas: 0,
    total: 0
  };
}

//CONTAR LLAMADAS POR CLASIFICACION Y POR OPERADOR
for (const llamada of this.llamadas) {
  if (estadisticasPorOperador[llamada.idOperador]) {
    estadisticasPorOperador[llamada.idOperador].total++;

    if (llamada.clasificacion === "Buena") {
      estadisticasPorOperador[llamada.idOperador].buenas++;
    } else if (llamada.clasificacion === "Media") {
      estadisticasPorOperador[llamada.idOperador].medias++;
    } else if (llamada.clasificacion === "Mala") {
      estadisticasPorOperador[llamada.idOperador].malas++;
    }
  }
}
```

También se realiza el código para mostrar los resultados en general de todos los operadores según su clasificación.

```
//TOTAL GENERAL
let buenasTotal = 0;
let mediasTotal = 0;
let malasTotal = 0;

for (const llamada of this.llamadas) {
  if (llamada.clasificacion === "Buena") buenasTotal++;
  else if (llamada.clasificacion === "Media") mediasTotal++;
  else if (llamada.clasificacion === "Mala") malasTotal++;
}
```

Menú Principal

El código muestra las opciones disponibles en la interfaz de consola. Presenta un menú numerado para que el usuario seleccione la acción.

```
//MENU PRINCIPAL
mostrarMenu(){
  console.log("\n=====MENU PRINCIPAL=====");
  console.log("1. Cargar registros de llamadas");
  console.log("2. Exportar historial de llamadas");
  console.log("3. Exportar listado de operadores");
  console.log("4. Exportar listado de clientes");
  console.log("5. Exportar rendimiento de operadores");
  console.log("6. Mostrar porcentaje de clasificacion de llamadas");
  console.log("7. Mostrar cantidad de llamadas por calificacion");
  console.log("8. Salir");
  console.log("=====");
}
```

Posteriormente se crea las funciones para manejar el menú con ayuda de un `switch()` para la opción seleccionada por el usuario, devolviendo las funciones vistas anteriormente.

```
switch (opcion){
  case "1":
    rl.question("Ingrese la ruta del archivo CSV: ", (ruta) => {
      callCenter.cargarArchivo(ruta);
      preguntarOpcion();
    });
    break;

  case "2":
    callCenter.exportarHistorialHTML();
    preguntarOpcion();
    break;

  case "3":
    callCenter.exportarOperadoresHTML();
    preguntarOpcion();
    break;

  case "4":
    callCenter.exportarClientesHTML();
    preguntarOpcion();
    break;

  case "5":
    callCenter.exportarRendimientoHTML();
    preguntarOpcion();
    break;

  case "6":
    callCenter.mostrarPorcentaje();
    preguntarOpcion();
}
```

En caso de que la opción seleccionada no esté definida, se realizó un código para dar error:

```
default:
  console.log("Opcion no valida, intente de nuevo");
  preguntarOpcion();
  break;
```

Ejecución del programa

Es el punto de entrada de la aplicación. Verifica si el archivo se ejecuta directamente, si es así, llama a la función main() para iniciar la aplicación.

```
//PARA EJECUTAR
if (require.main === module){
  main();
}

module.exports = CallCenter;
```

Reportes.js

generarHistorial(llamadas)

Genera un reporte HTML completo con el historial de todas las llamadas procesadas. Verifica que existan llamadas para exportar e itera sobre cada llamada creando filas HTML.

```
static generarHistorial(llamadas){
  if(llamadas.length === 0){
    throw new Error("No hay llamadas para exportar");
  }

  let filasTabla = "";

  for(const llamada of llamadas){
    filasTabla += `
    <tr>
      <td>${llamada.idOperador}</td>
      <td>${llamada.nombreOperador}</td>
      <td>${llamada.calificacion}</td>
      <td>${llamada.clasificacion}</td>
      <td>${llamada.idCliente}</td>
      <td>${llamada.nombreCliente}</td>
    </tr>`;
  }
}
```

generarOperadores(operadores)

Genera un reporte HTML con la información detallada de todos los operadores, estos se ordenan por ID de forma ascendente. Muestra la cantidad de llamadas atendidas por cada operador.

```
static generarOperadores(operadores){
  if(operadores.size === 0){
    throw new Error("No hay operadores para exportar");
  }

  const operadoresArray = Array.from(operadores.values());
  operadoresArray.sort((a, b) => a.id - b.id);

  let filasTabla = "";

  for(const operador of operadoresArray){
    filasTabla += `
    <tr>
      <td>${operador.id}</td>
      <td>${operador.nombre}</td>
      <td>${operador.llamadasAtendidas}</td>
    </tr>`;
  }

  const html = `<!DOCTYPE html>`
}
```

generarClientes(clientes)

Genera un reporte HTML con el listado completo de clientes, estos son ordenados por ID numérico. Solo incluye el ID y nombre del cliente, mostrando el número total de clientes registrados.

```
static generarClientes(clientes){
  if(clientes.size === 0){
    throw new Error("No hay clientes para exportar");
  }

  const clientesArray = Array.from(clientes.values());
  clientesArray.sort((a, b) => a.id - b.id);

  let filasTabla = "";

  for(const cliente of clientesArray) {
    filasTabla += `
    <tr>
      <td>${cliente.id}</td>
      <td>${cliente.nombre}</td>
    </tr>`;
  }
}
```

generarRendimientoOp()

Genera un reporte analítico del rendimiento de cada operador. Clacula la cantidad absoluta por operador y su porcentaje con la siguiente fórmula:
 $(\text{llamadas_operador} / \text{total_global}) * 100$

```
static generarRendimientoOp(operadores, totalLlamadasGlobales){
  if(operadores.size === 0){
    throw new Error("No hay operadores para exportar");
  }

  const operadoresArray = Array.from(operadores.values());
  operadoresArray.sort((a, b) => a.id - b.id);

  let filasTabla = "";

  for(const operador of operadoresArray){
    const porcentajeAtencion = totalLlamadasGlobales > 0
      ? ((operador.llamadasAtendidas / totalLlamadasGlobales) * 100).toFixed(2) : "0.00";

    filasTabla += `
    <tr>
      <td>${operador.id}</td>
      <td>${operador.nombre}</td>
      <td>${operador.llamadasAtendidas}</td>
      <td>${porcentajeAtencion}%</td>
    </tr>`;
  }
}
```

guardarHTML(nombreArchivo, contenido)

Este código guarda el contenido HTML de cada reporte generado en archivos físicos.

```
static guardarHTML(nombreArchivo, contenido){  
  try{  
    fs.writeFileSync(nombreArchivo, contenido, "utf-8");  
    return true;  
  } catch (error){  
    throw new Error("Error al guardar el archivo");  
  }  
}
```

Estructura HTML

Plantilla

Generación mediante template strings de JavaScript. Se realizó en cada reporte una tabla con los datos especificados, agregando los bordes y fuente de letras, agregando encabezados con el siguiente código en el <head>:

```
<head>
  <meta charset="UTF-8">
  <title>Rendimiento de Operadores</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 20px; }
    h1 { text-align: center; margin-bottom: 20px; }
    table { width: 100%; border-collapse: collapse; }
    th, td { border: 1px solid #000; padding: 8px; text-align: left; }
    th { background-color: #f2f2f2; }
  </style>
</head>
```

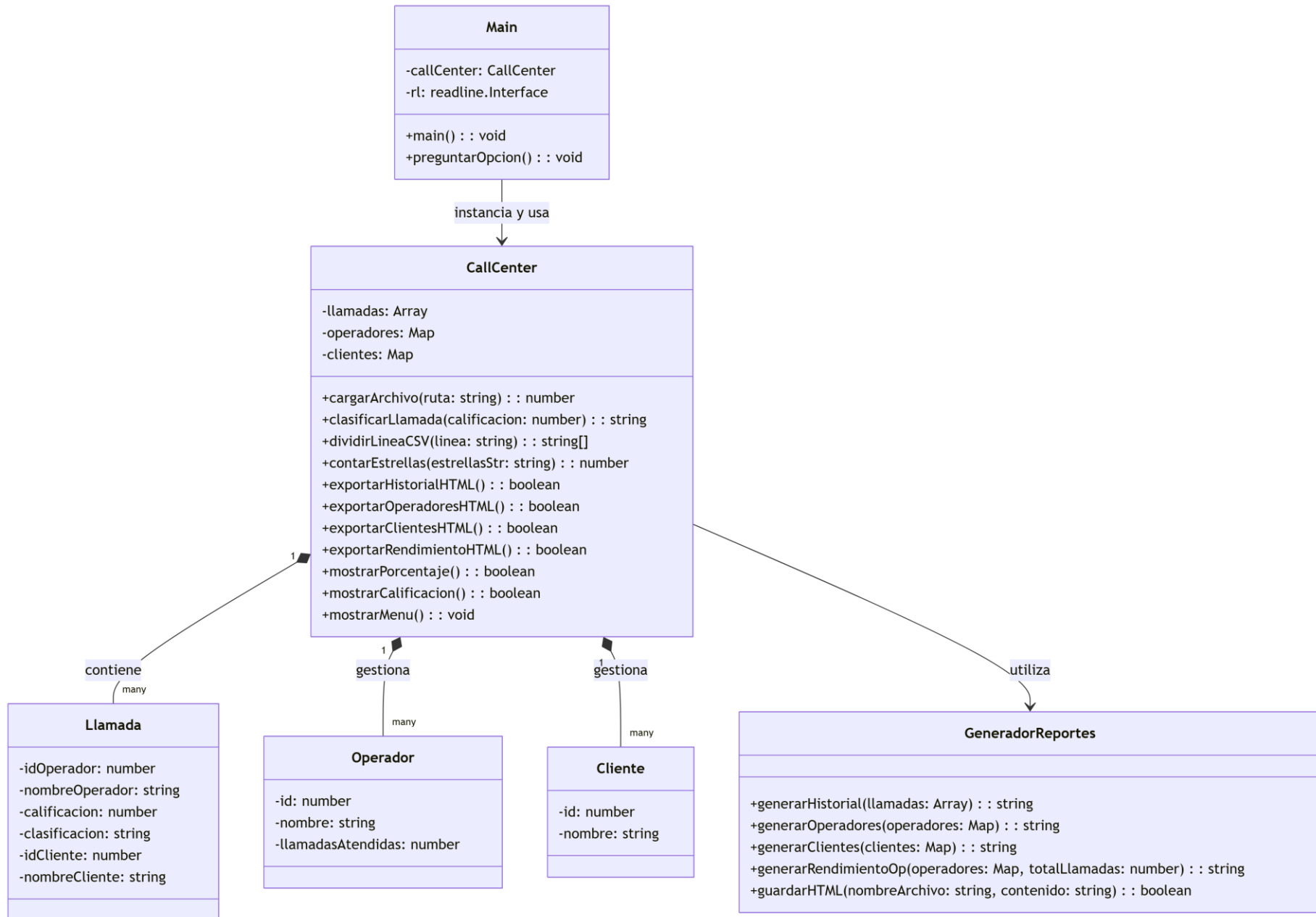
Datos

Con ayuda de <h1> es posible agregar el titulo del reporte, posteriormente con <th> se agrega el encabezado de cada columna de la tabla de la siguiente manera:

```
<body>
  <h1>Listado de Operadores</h1>
  <table>
    <thead>
      <tr>
        <th>ID Operador</th>
        <th>Nombre Operador</th>
        <th>Llamadas atendidas</th>
      </tr>
    </thead>
    <tbody>
      ${filasTabla}
    </tbody>
  </table>
  <p>Total de operadores: ${operadores.size}</p>
</body>
</html>`;
```

DIAGRAMA DE FLUJO

DIAGRAMA DE CLASES



CONSIDERACIONES TÉCNICAS

¿Qué formato debe tener el archivo csv para ser procesado?

El archivo debe tener 5 columnas en el siguiente orden:

- ID_Operador (número entero)
- Nombre_Operador (texto, puede ir entre comillas si este tiene comas)
- Estrellas (cadena en formato "X;X;X;X;X" donde X representa cada estrella.
- ID_Cliente (número entero)
- Nombre_Cliente (texto, puede ir entre comillas si este tiene comas)

¿Qué pasa si el archivo csv no cumple con el formato requerido?

El sistema procesará las líneas válidas e ignorará aquellas con formatos incorrectos, mostrando mensajes de error específicos por línea.

¿El sistema requiere encabezados en el archivo csv?

Sí, la primera línea debe contener los encabezados. El sistema detecta automáticamente si la primera línea contiene "id_operador" y la omite en el procesamiento.

¿El sistema depende de paquetes externos?

No, utiliza únicamente módulos nativos de Node.js (fs y readline), por lo que no requiere instalación de dependencias adicionales.

¿Los reportes incluyen toda la información procesada?

Sí, los reportes HTML, contienen tablas completas con todos los datos cargador y calculados, incluyendo totales y porcentajes